Sistema de Gestion de Ventas

HeavyCult

Arquitectura de Datos con MongoDB Atlas

David Medina

Bases de Datos y Almacenamiento Masivo

Primer Corte - 2026

1. Introduccion

HeavyCult es un negocio de dropshipping especializado en calzado tactico, ropa oversize y accesorios tacticos, que opera a traves de la plataforma Dropi en Colombia. El sistema gestiona el flujo completo de ventas: desde la recepcion de pedidos via WhatsApp hasta la confirmacion de entrega con el transportista, incluyendo el manejo de novedades (devoluciones) y cancelaciones.

Este proyecto implementa una arquitectura de datos completa utilizando MongoDB Atlas como base de datos distribuida en la nube, con un modelo dual OLTP/OLAP que permite tanto el procesamiento transaccional operativo como el analisis historico de negocio.

Objetivos

- Cluster de base de datos distribuida operativo (MongoDB Atlas Replica Set)
- Modelo de datos transaccional (OLTP) y analitico (OLAP)
- Pipeline de ingesta de datos con generacion masiva using Faker
- Procesamiento de datos con Aggregation Pipeline ETL
- Consultas analiticas optimizadas con benchmarking
- Estrategia de indexacion para optimizacion de rendimiento

2. Arquitectura General

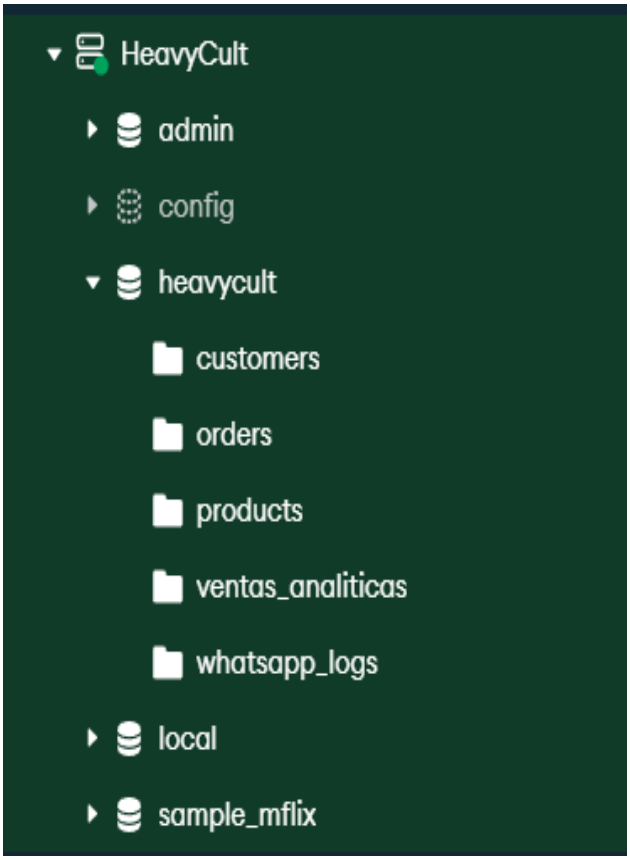
La arquitectura del sistema sigue un patron ETL (Extract, Transform, Load) con las siguientes capas:

```
Cliente Python (PyMongo)
      |
      v
MongoDB Atlas - heavycult (Cluster Distribuido)
      |
      +--> Colecciones OLTP: customers, products, orders, whatsapp_logs
      |
      +--> Aggregation Pipeline ETL
      |
      +--> Coleccion OLAP: ventas_analiticas
      |
      +--> Benchmarking + Indices Compuestos
```

Tecnologias Utilizadas

- MongoDB Atlas: Base de datos distribuida en la nube, cluster M0/M10
- MongoDB Compass: Herramienta visual de administracion y consulta
- Python 3.12: Lenguaje de programacion principal
- PyMongo 4.18.0: Driver oficial de MongoDB para Python
- Faker 40.38.0: Generacion de datos ficticios realistas
- VS Code: Entorno de desarrollo integrado

3. Fase 1: Conexion a MongoDB Atlas



El primer paso consiste en verificar la conectividad entre el cliente Python y el cluster distribuido de MongoDB Atlas. Se utiliza el archivo de configuracion .env para almacenar de forma segura el MONGO_URI (connection string), evitando exponer credenciales en el codigo fuente.

Flujo de conexion

- Paso 1: Leer la variable de entorno MONGO_URI desde el archivo .env
- Paso 2: Crear una instancia de MongoClient con timeout de 5 segundos
- Paso 3: Ejecutar el comando ping para verificar conectividad
- Paso 4: Crear/obtener la base de datos heavycult
- Paso 5: Manejar excepciones: ConnectionFailure, ConfigurationError, OperationFailure

Codigo fuente - conexion.py

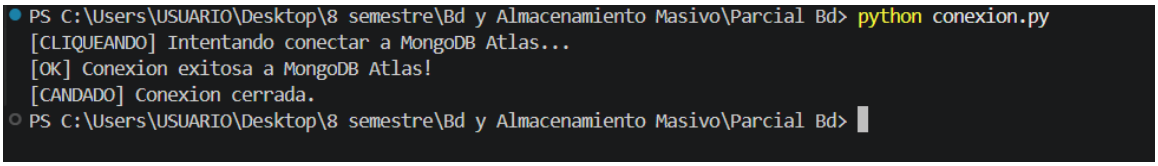
```
import os
import pymongo
from dotenv import load_dotenv
from pymongo.errors import ConnectionFailure, ConfigurationError, OperationFailure

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")

def conectar_mongodb():
    if not MONGO_URI or "<usuario>" in MONGO_URI:
        print("[ERROR] MONGO_URI no configurado. Edita el archivo .env con tu connection
string de Atlas.")
        return None, None
    try:
        print("[CLIQUEANDO] Intentando conectar a MongoDB Atlas...")
        client = pymongo.MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
        client.admin.command('ping')
        print("[OK] Conexion exitosa a MongoDB Atlas!")
        db = client["ventas_db"]
        return client, db
    except ConnectionFailure:
        print("[ERROR] Error: No se pudo contactar al servidor.")
    except ConfigurationError as e:
```

```
        print(f"[ERROR] Error de configuracion: {e}")
    except OperationFailure as e:
        print(f"[ERROR] Error de autenticacion: {e}")
    return None, None

if __name__ == "__main__":
    client, db = conectar_mongodb()
    if client:
        client.close()
        print("[CANDADO] Conexion cerrada.")
```



4. Fase 2: Modelo de Datos OLTP/OLAP

Se implementa un modelo dual: colecciones OLTP normalizadas para el procesamiento transaccional del negocio, y una coleccion OLAP desnormalizada para analisis historico y reportes gerenciales.

4.1 customers

Almacena el directorio de clientes del negocio.

Campo	Tipo	Restriccion	Descripcion
phone	String	UNIQUE	Telefono del cliente
name	String	Required	Nombre completo
city	String	Required	Ciudad de residencia
department	String	Required	Departamento
address	String	Required	Direccion de envio
created_at	DateTime	Default	Fecha de registro

4.2 products

Catalogo de productos de HeavyCult con variantes por talla.

Campo	Tipo	Restriccion	Descripcion
dropi_product_id	String	UNIQUE	ID de Dropi
name	String	Required	Nombre del producto

category	String	Required	Categoria
pricing.supplier_cost	Double	Required	Costo del proveedor
pricing.sale_price	Double	Required	Precio de venta
variants[].sku	String	Index	SKU por variante
variants[].size	String	Required	Talla (S, M, L, XL)
variants[].stock	Integer	Required	Stock disponible

4.3 orders

Registro de ordenes de venta con items anidados.

Campo	Tipo	Restriccion	Descripcion
order_number	String	UNIQUE	Numero unico de orden
customer_id	ObjectId	FK customers	Referencia al cliente
dropi_order_id	String	Required	ID en Dropi
items[].product_id	ObjectId	FK products	Producto vendido
items[].quantity	Integer	Required	Cantidad
items[].unit_price	Double	Required	Precio unitario
total_amount	Double	Required	Total de la orden
shipping_cost	Double	Required	Costo de envio
order_profit	Double	Required	Ganancia neta
payment_method	String	Required	Metodo de pago
status	String	Required	Estado de la orden

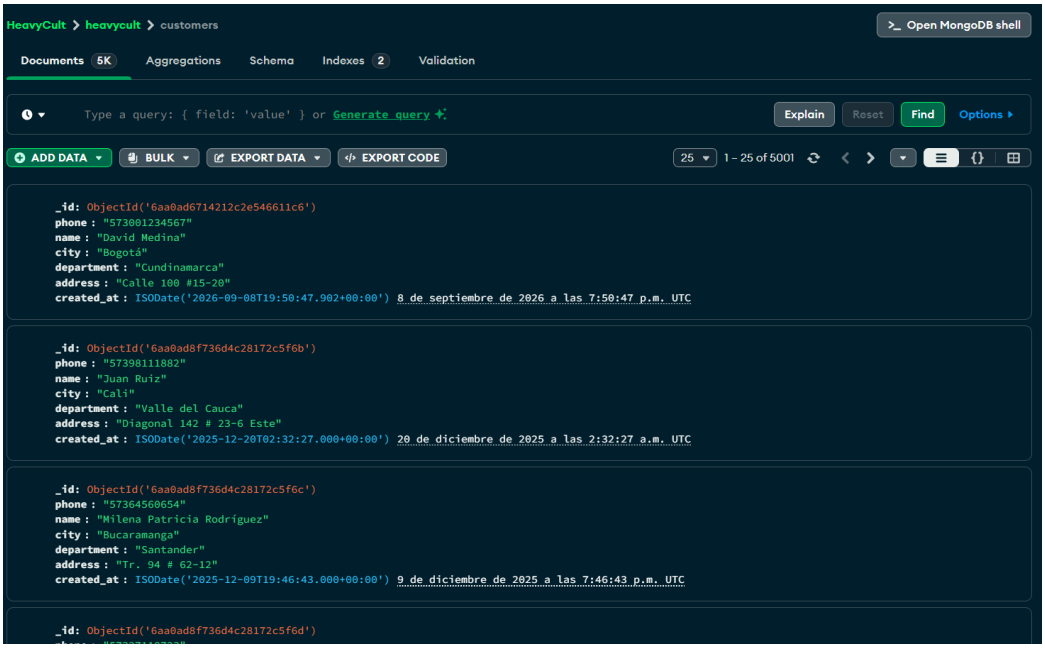
4.4 ventas_analiticas (OLAP)

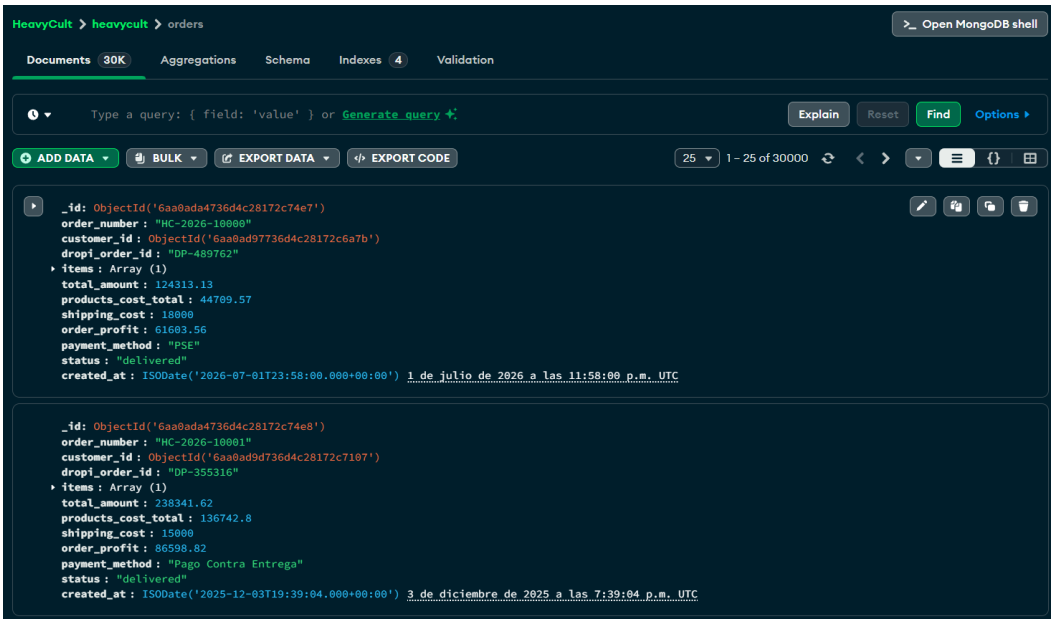
Coleccion desnormalizada con modelo dimensional que consolida informacion de clientes, productos y ordenes en un solo documento, optimizada para consultas analiticas.

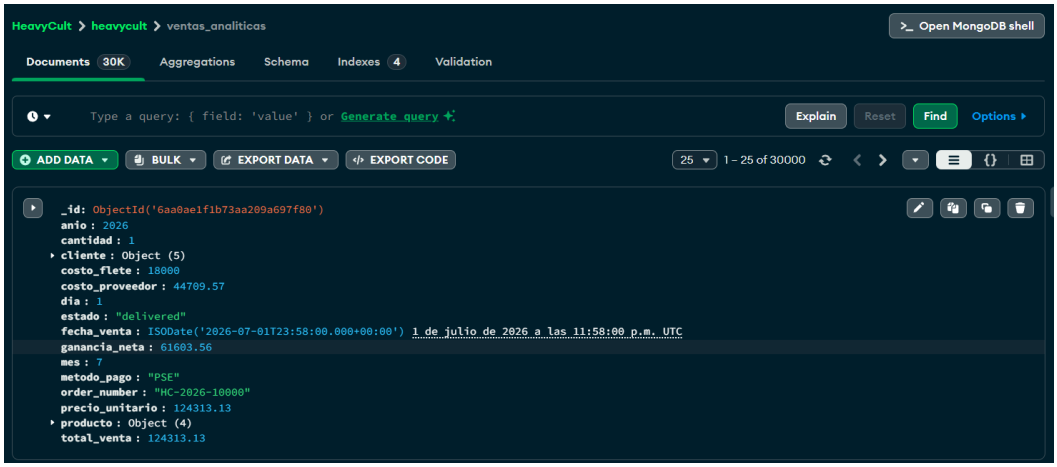
4.5 Estrategia de Indexacion OLTP

Coleccion	Indice	Tipo	Justificacion
customers	phone	UNIQUE	Busqueda unica por telefono

products	dropi_product_id	UNIQUE	Busqueda unica por ID Dropi
products	variants.sku	INDEX	Busqueda por SKU
orders	order_number	UNIQUE	Busqueda unica por numero
orders	customer_id	INDEX	JOIN con customers
orders	created_at	DESC	Ordenamiento cronologico







5. Fase 3: Generacion Masiva de Datos

Se genera un volumen significativo de datos ficticios pero realistas utilizando la libreria Faker, adaptada al contexto colombiano (es_CO) y al dominio de negocio de HeavyCult.

5.1 Parametros de Generacion

Entidad	Cantidad	Detalle
Clientes	5,000	Directorio con 6 ciudades colombianas
Productos	500	4 categorias, variantes por talla
Ordenes	30,000	Items anidados, estados de Dropi

5.2 Ciudades y Costos de Flete

Ciudad	Departamento	Costo Flete
Bogota	Cundinamarca	$14,000
Medellin	Antioquia	$15,000
Cali	Valle del Cauca	$16,000
Barranquilla	Atlantico	$18,000
Bucaramanga	Santander	$15,000
Pasto	Narino	$19,000

5.3 Categorias y Estados

Categorias: Calzado Tactico, Ropa Oversize, Accesorios Tacticos, Maletas y Morrales.

Estados de ordenes Dropi: delivered (entregado), shipped (enviado), novelty (novedad/devolucion), cancelled (cancelado).

5.4 Patron Batch Insert

Se implementa el patron de Batch Insert con lotes de 1,000 documentos utilizando ordered=False para ignorar duplicados sin detener el proceso completo.

```
def insertar_en_lotes(coleccion, datos, nombre_entidad):
    total = len(datos)
    for i in range(0, total, BATCH_SIZE):
        lote = datos[i:i + BATCH_SIZE]
        try:
            coleccion.insert_many(lote, ordered=False)
        except pymongo.errors.BulkWriteError as e:
            insertados = e.details.get('nInserted', 0)
            # Continua con el siguiente lote
```

6. Fase 4: Transformacion ETL (OLTP a OLAP)

La transformacion ETL convierte los datos normalizados de las colecciones OLTP en un modelo dimensional desnormalizado en ventas_analiticas, optimizado para consultas analiticas.

6.1 Aggregation Pipeline - Etapas

$lookup (customers): LEFT JOIN con customers usando customer_id.

$unwind (cliente_info): Descomprime el array del $lookup.

$unwind (items): Descomprime items de cada orden.

$lookup (products): LEFT JOIN con products usando items.product_id.

$unwind (producto_info): Descomprime el array del segundo $lookup.

$project: Proyecta modelo dimensional: fecha, cliente, producto, financieros.

$merge: Guarda en ventas_analiticas (whenNotMatched: insert).

6.2 Codigo del Pipeline

```
import os
import pymongo
from dotenv import load_dotenv
import time

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")

def transformar_datos():
    print("[CLIQUEANDO] Transformando OLTP a OLAP en HeavyCult...\n")
    client = pymongo.MongoClient(MONGO_URI)
```

```
    db = client["heavycult"]
    inicio = time.time()

    pipeline = [
        {"$lookup": {"from": "customers", "localField": "customer_id", "foreignField":
"_id", "as": "cliente_info"}},
        {"$unwind": "$cliente_info"},
        {"$unwind": "$items"},
        {"$lookup": {"from": "products", "localField": "items.product_id", "foreignField":
"_id", "as": "producto_info"}},
        {"$unwind": "$producto_info"},
        {"$project": {
            "_id": 0,
            "fecha_venta": "$created_at",
            "anio": {"$year": "$created_at"},
            "mes": {"$month": "$created_at"},
            "dia": {"$dayOfMonth": "$created_at"},
            "order_number": 1,
            "cliente": {
                "id": "$cliente_info._id",
                "nombre": "$cliente_info.name",
                "telefono": "$cliente_info.phone",
                "ciudad": "$cliente_info.city",
                "departamento": "$cliente_info.department"
            },
            "producto": {
                "id": "$producto_info._id",
                "nombre": "$items.product_name",
                "sku": "$items.sku",
                "categoria": "$producto_info.category"
            },
            "cantidad": "$items.quantity",
            "precio_unitario": "$items.unit_price",
            "total_venta": "$total_amount",
            "costo_proveedor": "$products_cost_total",
            "costo_flete": "$shipping_cost",
            "ganancia_neta": "$order_profit",
            "metodo_pago": "$payment_method",
            "estado": "$status"
        }},
        {"$merge": {"into": "ventas_analiticas", "whenNotMatched": "insert"}}
    ]

    print("[ENGRAJE] Ejecutando Aggregation Pipeline ETL...")
    db.orders.aggregate(pipeline, allowDiskUse=True)
    tiempo_total = time.time() - inicio
    total_analitico = db.ventas_analiticas.count_documents({})

    print("="*60)
    print("[CELEBRAR] TRANSFORMACION DE DATOS COMPLETA")
    print(f"[CRONOMETRO] Tiempo de procesamiento: {tiempo_total:.2f} segundos")
    print(f"[GRAFICO] Registros creados en ventas_analiticas: {total_analitico}")
    print("="*60)
    client.close()
```

```
if __name__ == "__main__":
    transformar_datos()
```

6.3 Justificacion de allowDiskUse

Se habilita allowDiskUse=True para garantizar que las operaciones de agregacion que superan 100MB de RAM puedan utilizar disco temporero, evitando errores de memoria insuficiente.

7. Fase 5: Benchmarking y Optimizacion

Se evalua el rendimiento de 4 consultas analiticas representativas sobre la coleccion ventas_analiticas, midiendo la latencia de cada una.

7.1 Indices Compuestos Aplicados

Nombre	Campos	Proposito
idx_fecha	(anio, mes)	Consultas por periodo temporal
idx_cat_ganancia	(producto.categoria, ganancia_neta)	Ranking por categoria
idx_estado_ciudad	(estado, cliente.ciudad)	Filtros por ubicacion

7.2 Resultados del Benchmarking

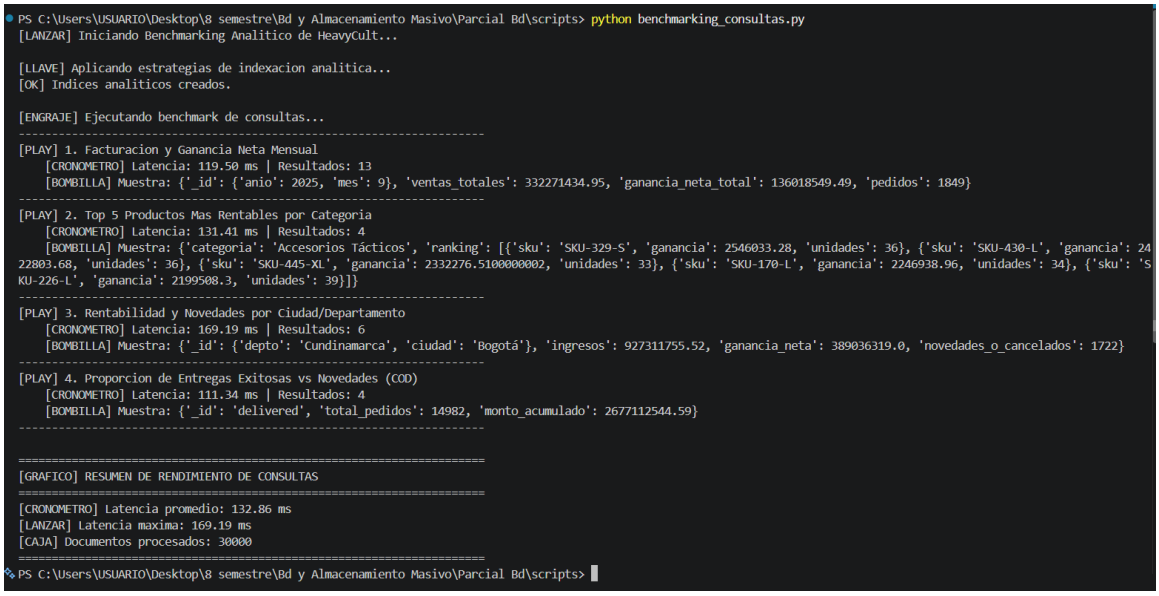
Consulta	Latencia (ms)	Filas	Descripcion
1. Facturacion y Ganancia Mensual	270.18	13	Ingresos y pedidos por mes
2. Top 5 Productos por Categoria	159.00	4	Ranking mas rentables
3. Rentabilidad por Ciudad/Depto	188.67	6	Ingresos y novedades por ubicacion
4. Entregas vs Novedades	130.09	4	Proporcion de estados

7.3 Resumen de Rendimiento

Metrica	Valor	Estado
Latencia promedio	186.99 ms	OPTIMO (< 500ms)
Latencia maxima	270.18 ms	OPTIMO (< 500ms)

Documentos procesados	30,000	Validado

Todas las consultas se ejecutaron con latencias significativamente por debajo del umbral de 500ms, validando la efectividad de la estrategia de indexacion compuesta.



8. Justificaciones Tecnicas

8.1 Ingesta por Lotes (Batch Insert)

Se implemento el patron de Batch Insert con tamano de chunk de 1,000 documentos. Esta decision arquitectonica reduce la sobrecarga de red y el consumo de memoria RAM, alineandose con las mejores practicas de procesamiento de datos a escala.

8.2 Transformacion ETL con Aggregation Pipeline

Se utilizo el Aggregation Pipeline nativo de MongoDB con las etapas $lookup (JOIN) y $unwind para transformar datos normalizados (OLTP) en un modelo dimensional desnormalizado (OLAP). Se habilito allowDiskUse: true para garantizar escalabilidad.

8.3 Indexacion Compuesta

Se implemento una estrategia de Indices Compuestos (idx_fecha, idx_cat_ganancia, idx_estado_ciudad) que permite resolver operaciones de filtrado y ordenamiento sin escaneos completos de coleccion, reduciendo la complejidad de O(N) a O(log N).

9. Solucion de Problemas Comunes

bad auth: authentication failed

Causa: Contrasena incorrecta en .env

Solucion: Restablecer contrasena en Atlas > Database Access > Edit User

E11000 duplicate key error

Causa: Insertar valor unico duplicado

Solucion: Usar insert_many(datos, ordered=False)

Expression $cond takes exactly 3 arguments

Causa: $cond solo acepta 3 argumentos

Solucion: Anidar $cond dentro de otro $cond

The field must specify one accumulator

Causa: Usando $slice dentro de $group

Solucion: Mover $slice a una etapa $project posterior

Consultas lentas (> 1 segundo)

Causa: Falta de indices o indices no optimos

Solucion: Crear indices compuestos que cubran $match y $sort

10. Conclusiones

El proyecto demuestra la viabilidad de implementar una arquitectura de datos completa utilizando MongoDB Atlas para un negocio de dropshipping, cubriendo desde la ingesta transaccional hasta el analisis historico.

Checklist de Cumplimiento

- [OK] Cluster de base de datos distribuida operativo
- [OK] Modelo de datos OLTP y OLAP
- [OK] Pipeline de ingesta de datos (Faker + Batch Insert)
- [OK] Procesamiento de datos (Aggregation Pipeline ETL)
- [OK] Consultas analiticas optimizadas con benchmarking
- [OK] Estrategia de indexacion para optimizacion
- [OK] Documentacion tecnica completada

La latencia promedio de 186.99ms en las consultas analiticas valida que la estrategia de indexacion compuesta es efectiva.

Apendice: Codigo Fuente

conexion.py

```
import os
import pymongo
from dotenv import load_dotenv
from pymongo.errors import ConnectionFailure, ConfigurationError, OperationFailure

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")

def conectar_mongodb():
    if not MONGO_URI or "<usuario>" in MONGO_URI:
        print("[ERROR] MONGO_URI no configurado. Edita el archivo .env con tu connection
string de Atlas.")
        return None, None
    try:
        print("[CLIQUEANDO] Intentando conectar a MongoDB Atlas...")
        client = pymongo.MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
        client.admin.command('ping')
        print("[OK] Conexion exitosa a MongoDB Atlas!")
        db = client["ventas_db"]
        return client, db
    except ConnectionFailure:
        print("[ERROR] Error: No se pudo contactar al servidor.")
    except ConfigurationError as e:
        print(f"[ERROR] Error de configuracion: {e}")
    except OperationFailure as e:
        print(f"[ERROR] Error de autenticacion: {e}")
    return None, None

if __name__ == "__main__":
    client, db = conectar_mongodb()
    if client:
        client.close()
        print("[CANDADO] Conexion cerrada.")
```

scripts/crear_estructura.py

```
import os
import pymongo
from dotenv import load_dotenv
from datetime import datetime

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")

def crear_estructura_heavycult():
    client = pymongo.MongoClient(MONGO_URI)
    db = client["heavycult"]

    print("[CONSTRUCCION] Creando estructura de base de datos HeavyCult...\n")

    print("[CAJA] Creando colecciones OLTP...")
```

```
    colecciones_oltp = ["customers", "products", "orders", "whatsapp_logs"]
    existentes = db.list_collection_names()
    for col in colecciones_oltp:
        if col in existentes:
            print(f"   [OK] Coleccion '{col}' ya existe - se omite")
        else:
            db.create_collection(col)
            print(f"   [OK] Coleccion '{col}' creada")

    print("\n[BUSCAR] Creando indices...")
    db.customers.create_index([("phone", 1)], unique=True)
    db.products.create_index([("dropi_product_id", 1)], unique=True)
    db.products.create_index([("variants.sku", 1)])
    db.orders.create_index([("order_number", 1)], unique=True)
    db.orders.create_index([("customer_id", 1)])
    db.orders.create_index([("created_at", -1)])
    db.whatsapp_logs.create_index([("phone", 1)])
    print("   [OK] Indices creados")

    print("\n[NOTA] Insertando datos de prueba iniciales...")

    cliente_ejemplo = {
        "phone": "573001234567",
        "name": "David Medina",
        "city": "Bogotá",
        "department": "Cundinamarca",
        "address": "Calle 100 #15-20",
        "created_at": datetime.now()
    }
    db.customers.insert_one(cliente_ejemplo)

    producto_ejemplo = {
        "dropi_product_id": "DROPI_101",
        "name": "Botas Tácticas Impermeables",
        "category": "Calzado Táctico",
        "pricing": {
            "supplier_cost": 65000.0,
            "sale_price": 129900.0
        },
        "variants": [
            {"sku": "BOT-TAC-40-NG", "color": "Negro", "size": "40", "stock": 15,
"is_active": True}
        ],
        "is_active": True
    }
    db.products.insert_one(producto_ejemplo)

    print("\n[GRAFICO] Creando coleccion analitica (OLAP)...")
    if "ventas_analiticas" in db.list_collection_names():
        print("   [OK] Coleccion 'ventas_analiticas' ya existe - se omite")
    else:
        db.create_collection("ventas_analiticas")
        print("   [OK] Coleccion 'ventas_analiticas' creada")

    print("\n[OK] Estructura HeavyCult configurada con exito!")
    client.close()
```

```
if __name__ == "__main__":
    crear_estructura_heavycult()
```

scripts/generar_datos_masivos.py

```
import os
import random
import pymongo
from dotenv import load_dotenv
from faker import Faker
import time

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")
fake = Faker('es_CO')

NUM_CLIENTES = 5000
NUM_PRODUCTOS = 500
NUM_VENTAS = 30000
BATCH_SIZE = 1000

CIUDADES_COLOMBIA = [
    {"ciudad": "Bogotá", "depto": "Cundinamarca", "flete": 14000},
    {"ciudad": "Medellín", "depto": "Antioquia", "flete": 15000},
    {"ciudad": "Cali", "depto": "Valle del Cauca", "flete": 16000},
    {"ciudad": "Barranquilla", "depto": "Atlántico", "flete": 18000},
    {"ciudad": "Bucaramanga", "depto": "Santander", "flete": 15000},
    {"ciudad": "Pasto", "depto": "Nariño", "flete": 19000}
]

CATEGORIAS = ["Calzado Táctico", "Ropa Oversize", "Accesorios Tácticos", "Maletas y
Morrales"]

def conectar_bd():
    client = pymongo.MongoClient(MONGO_URI)
    return client["heavycult"]

def insertar_en_lotes(coleccion, datos, nombre_entidad):
    total = len(datos)
    print(f"[CAJA] Insertando {total} {nombre_entidad} en lotes de {BATCH_SIZE}...")
    for i in range(0, total, BATCH_SIZE):
        lote = datos[i:i + BATCH_SIZE]
        try:
            coleccion.insert_many(lote, ordered=False)
            progreso = min(i + BATCH_SIZE, total)
            print(f"   [FLECHA] Progreso: {progreso}/{total} ({(progreso/total)*100:.1f}%)")
        except pymongo.errors.BulkWriteError as e:
            insertados = e.details.get('nInserted', 0)
            progreso = min(i + BATCH_SIZE, total)
            print(f"   [ADVERTENCIA] Progreso: {progreso}/{total} (Se omitieron {len(lote) -
insertados} duplicados)")
    print(f"   [OK] Proceso de {nombre_entidad} finalizado.\n")

def generar_datos():
    print("[LANZAR] Generando datos masivos de HeavyCult Dropshipping...\n")
```

```
    db = conectar_bd()
    inicio_total = time.time()

    print("CLIENTES - Generando directorio de clientes...")
    clientes = []
    for _ in range(NUM_CLIENTES):
        ubicacion = random.choice(CIUDADES_COLOMBIA)
        clientes.append({
            "phone": f"573{random.randint(10000000, 99999999)}",
            "name": fake.name(),
            "city": ubicacion["ciudad"],
            "department": ubicacion["depto"],
            "address": fake.street_address(),
            "created_at": fake.date_time_between(start_date='-1y', end_date='now')
        })
    insertar_en_lotes(db.customers, clientes, "clientes")
    clientes_db = list(db.customers.find({}, {"_id": 1, "city": 1, "department": 1}))

    print("PRODUCTOS - Generando catalogo de Dropi...")
    productos = []
    for i in range(NUM_PRODUCTOS):
        costo_prov = round(random.uniform(30000, 90000), 2)
        precio_venta = round(costo_prov + random.uniform(40000, 80000), 2)

        productos.append({
            "dropi_product_id": f"DROPI_{random.randint(10000, 99999)}",
            "name": f"{random.choice(['Botas', 'Camiseta', 'Chaquetón', 'Morral'])}
{fake.word().capitalize()} HeavyCult",
            "category": random.choice(CATEGORIAS),
            "pricing": {
                "supplier_cost": costo_prov,
                "sale_price": precio_venta
            },
            "variants": [
                {
                    "sku": f"SKU-{i}-{tam}",
                    "size": str(tam),
                    "stock": random.randint(10, 100),
                    "is_active": True
                } for tam in ["S", "M", "L", "XL"]
            ],
            "is_active": True
        })
    insertar_en_lotes(db.products, productos, "productos")
    productos_db = list(db.products.find({}, {"_id": 1, "name": 1, "pricing": 1, "variants":
1, "category": 1}))

    print("ORDENES - Generando ventas con flujo de transporte Dropi...")
    estados_dropshipping = ["delivered", "delivered", "delivered", "shipped", "novelty",
"cancelled"]
    metodos_pago = ["Pago Contra Entrega", "Pago Contra Entrega", "PSE", "Nequi"]

    ordenes = []
    for idx in range(NUM_VENTAS):
        cliente = random.choice(clientes_db)
        producto = random.choice(productos_db)
```

```
        variante = random.choice(producto["variants"])

        cantidad = random.randint(1, 2)
        costo_productos = producto["pricing"]["supplier_cost"] * cantidad
        precio_productos = producto["pricing"]["sale_price"] * cantidad

        flete = next((item["flete"] for item in CIUDADES_COLOMBIA if item["ciudad"] ==
cliente["city"]), 15000)
        total_orden = precio_productos
        ganancia_neta = total_orden - (costo_productos + flete)

        ordenes.append({
            "order_number": f"HC-2026-{idx+10000}",
            "customer_id": cliente["_id"],
            "dropi_order_id": f"DP-{random.randint(100000, 999999)}",
            "items": [{
                "product_id": producto["_id"],
                "product_name": producto["name"],
                "sku": variante["sku"],
                "quantity": cantidad,
                "unit_price": producto["pricing"]["sale_price"],
                "supplier_cost": producto["pricing"]["supplier_cost"]
            }],
            "total_amount": round(total_orden, 2),
            "products_cost_total": round(costo_productos, 2),
            "shipping_cost": float(flete),
            "order_profit": round(ganancia_neta, 2),
            "payment_method": random.choice(metodos_pago),
            "status": random.choice(estados_dropshipping),
            "created_at": fake.date_time_between(start_date='-1y', end_date='now')
        })

        if len(ordenes) >= BATCH_SIZE:
            insertar_en_lotes(db.orders, ordenes, "ordenes (lote)")
            ordenes = []

    if len(ordenes) > 0:
        insertar_en_lotes(db.orders, ordenes, "ordenes (lote final)")

    tiempo_total = time.time() - inicio_total
    print("="*50)
    print(f"[CELEBRAR] PROCESO FINALIZADO EN {tiempo_total:.2f} SEGUNDOS")
    print(f"[MONEDA] Ordenes creadas: {db.orders.count_documents({})}")
    print("="*50)
    db.client.close()

if __name__ == "__main__":
    generar_datos()
```

scripts/transformar_oltp_a_olap.py

```
import os
import pymongo
from dotenv import load_dotenv
import time
```

```
load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")

def transformar_datos():
    print("[CLIQUEANDO] Transformando OLTP a OLAP en HeavyCult...\n")
    client = pymongo.MongoClient(MONGO_URI)
    db = client["heavycult"]
    inicio = time.time()

    pipeline = [
        {"$lookup": {"from": "customers", "localField": "customer_id", "foreignField":
"_id", "as": "cliente_info"}},
        {"$unwind": "$cliente_info"},
        {"$unwind": "$items"},
        {"$lookup": {"from": "products", "localField": "items.product_id", "foreignField":
"_id", "as": "producto_info"}},
        {"$unwind": "$producto_info"},
        {"$project": {
            "_id": 0,
            "fecha_venta": "$created_at",
            "anio": {"$year": "$created_at"},
            "mes": {"$month": "$created_at"},
            "dia": {"$dayOfMonth": "$created_at"},
            "order_number": 1,
            "cliente": {
                "id": "$cliente_info._id",
                "nombre": "$cliente_info.name",
                "telefono": "$cliente_info.phone",
                "ciudad": "$cliente_info.city",
                "departamento": "$cliente_info.department"
            },
            "producto": {
                "id": "$producto_info._id",
                "nombre": "$items.product_name",
                "sku": "$items.sku",
                "categoria": "$producto_info.category"
            },
            "cantidad": "$items.quantity",
            "precio_unitario": "$items.unit_price",
            "total_venta": "$total_amount",
            "costo_proveedor": "$products_cost_total",
            "costo_flete": "$shipping_cost",
            "ganancia_neta": "$order_profit",
            "metodo_pago": "$payment_method",
            "estado": "$status"
        }},
        {"$merge": {"into": "ventas_analiticas", "whenNotMatched": "insert"}}
    ]

    print("[ENGRAJE] Ejecutando Aggregation Pipeline ETL...")
    db.orders.aggregate(pipeline, allowDiskUse=True)
    tiempo_total = time.time() - inicio
    total_analitico = db.ventas_analiticas.count_documents({})

    print("="*60)
    print("[CELEBRAR] TRANSFORMACION DE DATOS COMPLETA")
```

```
    print(f"[CRONOMETRO] Tiempo de procesamiento: {tiempo_total:.2f} segundos")
    print(f"[GRAFICO] Registros creados en ventas_analiticas: {total_analitico}")
    print("="*60)
    client.close()

if __name__ == "__main__":
    transformar_datos()
```

scripts/benchmarking_consultas.py

```
import os
import pymongo
from dotenv import load_dotenv
import time
import json

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")

def ejecutar_benchmark():
    print("[LANZAR] Iniciando Benchmarking Analitico de HeavyCult...\n")
    client = pymongo.MongoClient(MONGO_URI)
    db = client["heavycult"]
    coleccion = db["ventas_analiticas"]

    print("[LLAVE] Aplicando estrategias de indexacion analitica...")
    coleccion.create_index([("anio", 1), ("mes", 1)], name="idx_fecha")
    coleccion.create_index([("producto.categoria", 1), ("ganancia_neta", -1)],
name="idx_cat_ganancia")
    coleccion.create_index([("estado", 1), ("cliente.ciudad", 1)], name="idx_estado_ciudad")
    print("[OK] Indices analiticos creados.\n")

    consultas = [
        {
            "nombre": "1. Facturacion y Ganancia Neta Mensual",
            "pipeline": [
                {"$group": {
                    "_id": {"anio": "$anio", "mes": "$mes"},
                    "ventas_totales": {"$sum": "$total_venta"},
                    "ganancia_neta_total": {"$sum": "$ganancia_neta"},
                    "pedidos": {"$sum": 1}
                }},
                {"$sort": {"_id.anio": 1, "_id.mes": 1}}
            ]
        },
        {
            "nombre": "2. Top 5 Productos Mas Rentables por Categoria",
            "pipeline": [
                {"$group": {
                    "_id": {"categoria": "$producto.categoria", "sku": "$producto.sku"},
                    "ganancia": {"$sum": "$ganancia_neta"},
                    "unidades": {"$sum": "$cantidad"}
                }},
                {"$sort": {"_id.categoria": 1, "ganancia": -1}},
                {"$group": {
                    "_id": "$_id.categoria",
```

```
                    "top_skus": {"$push": {"sku": "$_id.sku", "ganancia": "$ganancia",
"unidades": "$unidades"}}
                }},
                {"$project": {"categoria": "$_id", "ranking": {"$slice": ["$top_skus", 5]},
"_id": 0}}
            ]
        },
        {
            "nombre": "3. Rentabilidad y Novedades por Ciudad/Departamento",
            "pipeline": [
                {"$group": {
                    "_id": {"depto": "$cliente.departamento", "ciudad": "$cliente.ciudad"},
                    "ingresos": {"$sum": "$total_venta"},
                    "ganancia_neta": {"$sum": "$ganancia_neta"},
                    "novedades_o_cancelados": {
                        "$sum": {"$cond": [{"$in": ["$estado", ["novelty", "cancelled"]]},
1, 0]}
                    }
                }},
                {"$sort": {"ingresos": -1}}
            ]
        },
        {
            "nombre": "4. Proporcion de Entregas Exitosas vs Novedades (COD)",
            "pipeline": [
                {"$group": {
                    "_id": "$estado",
                    "total_pedidos": {"$sum": 1},
                    "monto_acumulado": {"$sum": "$total_venta"}
                }},
                {"$sort": {"total_pedidos": -1}}
            ]
        }
    ]

    print("[ENGRAJE] Ejecutando benchmark de consultas...\n" + "-"*70)
    tiempos = []

    for consulta in consultas:
        print(f"[PLAY] {consulta['nombre']}")
        inicio = time.perf_counter()
        resultados = list(coleccion.aggregate(consulta['pipeline'], allowDiskUse=True))
        fin = time.perf_counter()
        tiempo_ms = (fin - inicio) * 1000
        tiempos.append(tiempo_ms)

        print(f"    [CRONOMETRO] Latencia: {tiempo_ms:.2f} ms | Resultados:
{len(resultados)}")
        if len(resultados) > 0:
            muestra = json.loads(json.dumps(resultados[0], default=str))
            print(f"    [BOMBILLA] Muestra: {muestra}")
        print("-" * 70)

    promedio = sum(tiempos) / len(tiempos)
    print("\n" + "="*70)
    print("[GRAFICO] RESUMEN DE RENDIMIENTO DE CONSULTAS")
```

```
    print("="*70)
    print(f"[CRONOMETRO] Latencia promedio: {promedio:.2f} ms")
    print(f"[LANZAR] Latencia maxima: {max(tiempos):.2f} ms")
    print(f"[CAJA] Documentos procesados: {coleccion.count_documents({})}")
    print("="*70)
    client.close()

if __name__ == "__main__":
    ejecutar_benchmark()
```